

TOBACCOGUARD: A SATELLITE-BASED TOBACCO CROP HEALTH MONITORING SYSTEM USING MACHINE LEARNING

¹Prince Rupiya ²Mr Manyonganise

¹h220536m@hit.ac.zw

Harare Institute of Technology, Harare, Zimbabwe
Department of Information Technology, Harare Institute of Technology

Abstract—Tobacco is a critical export crop for Zimbabwe, yet smallholder farmers lack accessible tools for timely detection of crop stress. Manual scouting is labour-intensive and is typically too infrequent to prevent yield loss. This paper presents TobaccoGuard, a web-based platform that monitors tobacco crop health by combining free satellite imagery from Google Earth Engine (GEE) and the Copernicus Sentinel-2 archive with a machine learning classification pipeline. Farmers register geo-referenced field polygons on an interactive Leaflet map, after which the system automatically retrieves a 90-day multi-temporal NDVI time series, derives five agronomic features, and applies a multinomial logistic regression classifier to assign one of three health statuses: HEALTHY, MODERATE_STRESS, or HIGH_STRESS. The classifier achieved 97 % overall accuracy on a held-out test set of 122 samples, with per-class F1-scores of 0.99, 0.97, and 0.95 respectively. A rule-based recommendation engine translates classifier outputs into four targeted agronomic interventions. The system is implemented as a Next.js 14 full-stack application with SQLite persistence and executes ML inference natively in TypeScript at runtime, eliminating any dependency on a Python interpreter in production. Functional evaluation across eight end-to-end test scenarios confirmed correct behaviour throughout the pipeline. The prototype demonstrates that a deployable, zero-licensing-cost precision agriculture tool for smallholder tobacco farmers is achievable using freely available satellite and cloud infrastructure.

Index Terms—Normalized Difference Vegetation Index (NDVI), satellite remote sensing, Google Earth Engine, tobacco crop monitoring, logistic regression, Sentinel-2, precision agriculture, crop health classification, Zimbabwe

I. INTRODUCTION

Tobacco remains one of Zimbabwe’s most significant agricultural export commodities, contributing substantially to foreign currency earnings and rural livelihoods [1]. The country’s tobacco production relies heavily on smallholder farmers, whose plots typically range from 0.5 to 5 hectares and whose access to technical support is constrained by limited extension officer availability. These farmers currently rely on visual field inspection and periodic advisory visits to identify crop stress. This reactive approach means that water stress, nutrient

deficiency, and disease pressure are frequently detected only after visible leaf wilting or discolouration has occurred, by which point significant yield loss may already be irreversible [2].

Satellite remote sensing offers a scalable alternative that operates continuously and at near-zero marginal cost per additional field monitored. The Normalized Difference Vegetation Index (NDVI), computed from red and near-infrared reflectance bands as $(B8 - B4)/(B8 + B4)$, provides an objective, non-invasive proxy for canopy photosynthetic activity and has been validated as an early indicator of plant stress across a wide range of crops before visual symptoms become apparent [3]. The Sentinel-2 satellite constellation, operated by the European Space Agency under the Copernicus programme, delivers freely accessible 10-metre-resolution multispectral imagery with an approximate five-day revisit time, making regular field-level monitoring practical without recurring data licensing costs [4].

Despite these opportunities, commercially available precision agriculture platforms such as EOSDA Crop Monitoring and Cropio impose subscription fees that are prohibitive for smallholder farmers or the institutions that serve them. Open-source GIS tools such as QGIS and Google Earth desktop exist but require significant remote sensing expertise to operate, well beyond that of most extension officers or farmer cooperatives. There is therefore a clear gap for a low-cost, user-friendly, web-based monitoring tool tailored to the Zimbabwean smallholder tobacco context.

This paper presents TobaccoGuard, a prototype system that addresses this gap by integrating freely available satellite data retrieval, domain-specific feature engineering, and a lightweight machine learning classifier into a deployable web application. Our principal contributions are:

- 1) A full-stack satellite monitoring pipeline that retrieves and processes Sentinel-2 NDVI time series via the GEE API and derives agronomic features at field polygon granularity.
- 2) A multinomial logistic regression classifier trained offline

with scikit-learn and serialised to a JSON coefficient file, enabling native TypeScript inference with no Python dependency at runtime.

- 3) A rule-based recommendation engine that maps classifier outputs and feature values to four targeted field management interventions.
- 4) An end-to-end functional evaluation demonstrating correct pipeline behaviour across all eight test scenarios.

The remainder of this paper is structured as follows. Section II reviews related work on NDVI-based crop monitoring and machine learning for crop classification. Section III presents the system design including architecture, database model, and algorithm design. Section IV details the implementation. Section V reports evaluation results. Section VI discusses findings and limitations. Section VII concludes.

II. LITERATURE REVIEW

Remote sensing applications in agriculture have expanded considerably since the public release of large-scale satellite image archives. NDVI computed from Landsat and MODIS imagery has been applied for decades to estimate crop yield, biomass, and canopy cover [2]. The shift to Sentinel-2's 10-metre spatial resolution has more recently enabled monitoring at the level of individual smallholder fields, which was previously impractical with 30-metre or 250-metre resolution data.

A. NDVI-Based Crop Stress Detection

Several studies have validated NDVI time series from Sentinel-2 for crop stress detection. Delegido et al. demonstrated that multi-temporal NDVI profiles can reliably differentiate between irrigated and rain-fed maize fields at classification accuracies exceeding 90%, showing that trajectory shape rather than single-date index values carries most discriminative information [5]. Kussul et al. further showed that incorporating temporal trend features derived from NDVI time series substantially improves classification accuracy for smallholder crop mapping in Sub-Saharan Africa, where phenological timing is more variable than in large-scale mechanised farming systems [6]. These findings motivated the inclusion of NDVI slope (trend) and variance as explicit features in the present work rather than relying solely on the temporal mean.

Tobacco-specific remote sensing research remains limited compared to staple grains, but the general relationship between NDVI and leaf photosynthetic capacity is well-established across broad-leaved crop species. Below $NDVI \approx 0.35$, significant chlorophyll reduction associated with water stress or nutrient deficiency is expected; above $NDVI \approx 0.60$, canopy closure is typically adequate and the crop is unlikely to be in acute stress [2].

B. Machine Learning for Crop Classification

Machine learning approaches to remote sensing crop health classification span logistic regression, support vector machines, random forests, and deep convolutional networks. Kpienbaareh et al. conducted a comparative study of these classifiers for smallholder crop mapping in Ghana using Sentinel-1 and

TABLE I
COMPARATIVE SUMMARY (COMMERCIAL VS. PROTOTYPE SYSTEMS)

System	Data	AI/ML	Cost	Deploy
EOSDA Crop Mon.	S-2	Adv.	Paid	SaaS only
Cropio	Multi-sat.	Rule-based	Paid	SaaS only
Research ML tools	Various	RF/DL	Free	Notebook
This work	GEE	LR	Free	Web app

Sentinel-2 features and found that logistic regression achieved classification accuracy within 2% of more complex ensemble models when training sets contained fewer than 1 000 labelled samples and features were carefully engineered [7]. This result is consistent with the bias-variance tradeoff literature: with limited training data, high-capacity models such as gradient boosted trees tend to overfit, while a regularised linear classifier generalises more reliably.

The practical case for logistic regression in this project was further strengthened by the deployment constraint that ML inference must execute natively in a Node.js TypeScript environment without a Python subprocess. A logistic regression model reduces to a matrix-vector multiplication followed by softmax normalisation, which is straightforward to implement in any language from a serialised coefficient file. More complex models such as random forests would require a significantly larger runtime library (e.g., ONNX Runtime) and a more complex serialisation scheme.

C. Existing Monitoring Platforms

Table I summarises commercially available and prototype systems relevant to satellite-based crop monitoring. Commercial platforms such as EOSDA Crop Monitoring and Cropio offer rich analytical features but impose subscription licensing costs that make them inaccessible to smallholder farmers and the underfunded institutions that support them. Research-grade ML systems achieve high classification accuracy but are packaged as Python notebooks rather than deployable web applications. TobaccoGuard occupies the prototype tier: it is packaged as a deployable web application that uses real Sentinel-2 satellite data, runs ML inference at zero licensing cost, and requires no remote sensing expertise from its end users.

III. SYSTEM/SOLUTION DESIGN

The system is organised as a four-layer pipeline: (i) satellite data acquisition via GEE, (ii) a Next.js backend for request handling and persistence, (iii) an intelligence layer for feature engineering and ML inference, and (iv) a React-based presentation layer. Modularity is a deliberate design principle: individual layers can be upgraded independently. For example, replacing the GEE satellite service with a direct Sentinel API call requires no changes to the ML or UI layers.

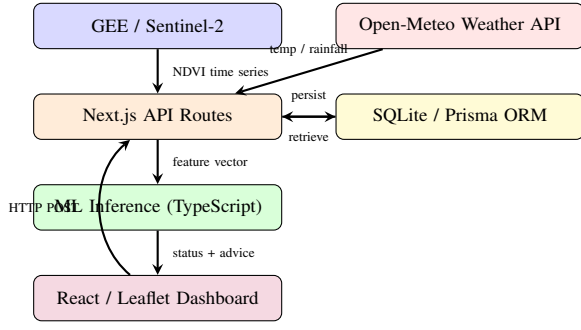


Fig. 1. Solution architecture: GEE and Open-Meteo supply external data to the Next.js backend, which drives ML inference and persists results to SQLite, while the React dashboard is the user-facing presentation layer.

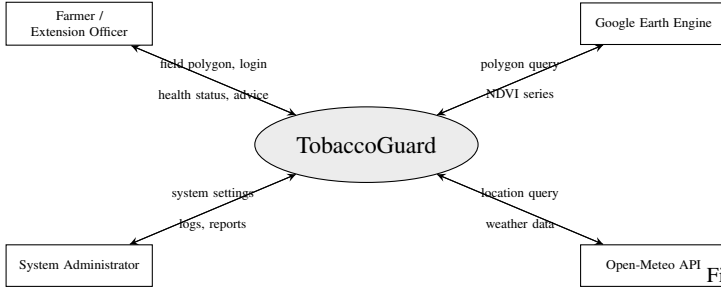


Fig. 2. Context diagram: TobaccoGuard interacts with farmers, the system administrator, GEE, and Open-Meteo.

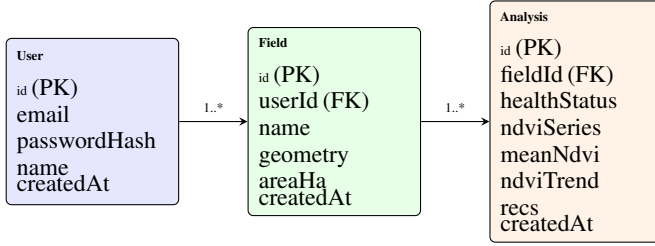


Fig. 3. Entity-relationship diagram. Each User owns one or more Fields, and each Field accumulates one or more Analysis records over time.

A. Solution Architecture

Figure 1 shows the solution architecture. Sentinel-2 NDVI data flows from GEE and 30-day weather summaries from Open-Meteo into the Next.js API, which constructs the feature vector, invokes the in-process ML inference module, persists the result, and returns structured JSON to the React dashboard.

B. Context and Data Flow

Figure 2 shows the system context. The primary stakeholders are farmers (or extension officers acting on their behalf) who register fields and view analysis results, and a system administrator who configures the deployment. The two external data services are the GEE API (Sentinel-2 NDVI) and the Open-Meteo REST API (temperature and rainfall).

C. Database Model

Figure 3 presents the entity-relationship diagram. Three entities are defined. *User* stores authentication credentials

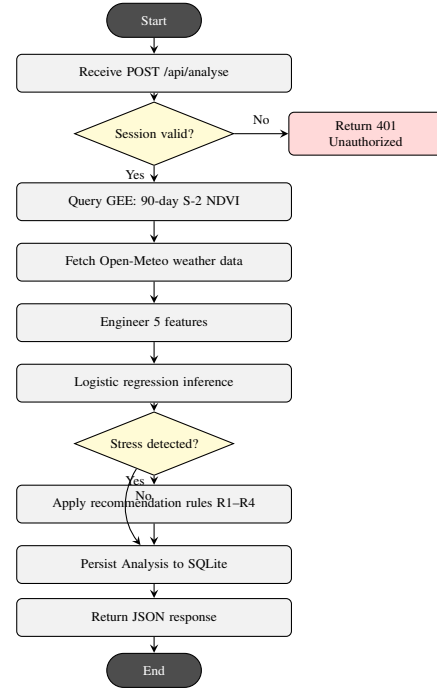


Fig. 4. Processing flow: from incoming HTTP request through GEE query, feature engineering, ML inference, recommendation generation, persistence, and HTTP response.

and profile information. *Field* stores the polygon geometry as GeoJSON, the Shoelace-computed area in hectares, and field metadata. *Analysis* stores each monitoring run's output: the serialised NDVI time series, derived scalar features, predicted health status, risk flags, and the generated recommendation list. The one-to-many cardinality from Field to Analysis enables trend tracking across successive monitoring runs of the same plot, which is important for detecting gradual decline patterns.

D. Algorithm Design

Figure 4 illustrates the processing flow. The five ordered steps of the classification pipeline are:

- 1) **Sentinel-2 retrieval:** Filter the COPERNICUS/S2_SR_HARMONIZED GEE collection to the polygon bounding box, the preceding 90 days, and cloud cover below 20%. Apply the QA60 bitmask cloud mask and compute mean NDVI per scene over the polygon at 10-metre resolution.
- 2) **Weather data:** Request 30-day temperature and precipitation summaries from the Open-Meteo /v1/archive endpoint for the polygon centroid coordinates.
- 3) **Feature engineering:** Derive five scalar features: `mean_ndvi` (temporal mean), `ndvi_trend` (OLS slope), `ndvi_variance` (temporal variance), `avg_temperature_c` (30-day mean), and `total_rainfall_mm` (30-day precipitation sum).
- 4) **ML inference:** Compute softmax logits over three classes using the pre-loaded coefficient matrix and return the argmax class with its probability vector.

TABLE II
DEVELOPMENT TOOLS AND TECHNOLOGIES

Component	Technology
Full-stack framework	Next.js 14 (React 18, TypeScript 5)
Mapping library	React-Leaflet 4, Leaflet-Draw
UI component library	shadcn/ui, Tailwind CSS 3
Data visualisation	Recharts 2 (AreaChart)
ORM / database	Prisma ORM 5 with SQLite
Satellite data	Google Earth Engine (service account)
Weather data	Open-Meteo REST API (free tier)
ML training (offline)	scikit-learn 1.4 (Python 3.11)
ML inference (runtime)	JSON coefficient file, TypeScript
Authentication	NextAuth.js 4 (credential provider)
Password hashing	bcryptjs

- 5) **Recommendation:** Evaluate four rule-based conditions on the classifier output and feature values to generate agronomic advice.

IV. IMPLEMENTATION

A. Development Environment and Technology Stack

The system was developed on Node.js 20 with Next.js 14 (App Router) as the full-stack framework. TypeScript is used throughout for static type safety. The Python scikit-learn library was used exclusively during the offline training phase; no Python interpreter is required at runtime. Table II lists the key technologies used in the project.

B. Frontend Implementation

The frontend consists of three primary views, illustrated as wireframes in Figs. 5 and 6.

The *Dashboard* view (Fig. 5) lists all registered fields with their most recent health status badge. Badges are colour-coded: green for HEALTHY, amber for MODERATE_STRESS, and red for HIGH_STRESS. Each field card displays the field name, area in hectares, and the date of the last analysis, enabling the farmer or extension officer to identify which fields are overdue for re-evaluation.

The *Create Field* view embeds a Leaflet map loaded with ESRI World Imagery satellite tiles and Leaflet-Draw restricted to polygon mode. As the user completes the polygon, the Shoelace (surveyor's) formula is applied client-side to compute the enclosed area from the coordinate ring, so the farmer receives immediate area feedback without a server round-trip.

The *Analysis Results* view (Fig. 6) renders the full output of one analysis run: a Recharts AreaChart of the 90-day NDVI time series with a reference line at NDVI=0.35, a health status badge with per-class probability breakdown, a risk flag panel surfacing any triggered recommendation rules, and an ordered list of agronomic interventions.

C. Backend API

Three API route groups handle system functionality. Authentication routes (`/api/auth`) implement credential-based login

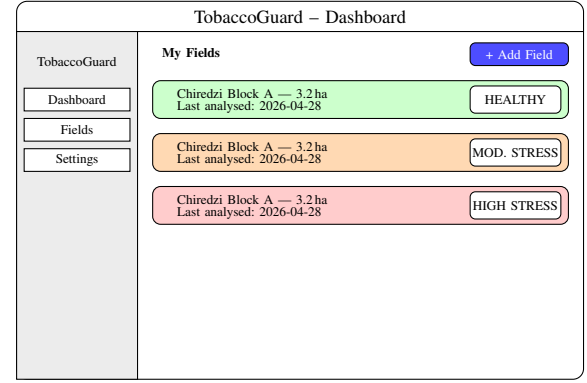


Fig. 5. Dashboard wireframe: field list with colour-coded health status badges and a quick-access Add Field button.

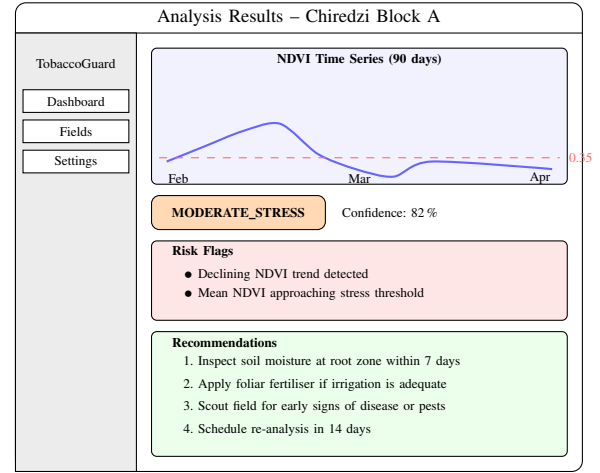


Fig. 6. Analysis Results wireframe: NDVI time series chart (top), health status badge with confidence score (middle), risk flags, and ordered recommendation list (bottom).

using NextAuth.js with bcrypt password hashing; passwords are never stored in plaintext and session tokens are kept server-side in the database. Field routes (`/api/field`) support creation, retrieval, and deletion of field records, with GeoJSON geometry validated on ingestion and area computed server-side as a consistency check against the client-computed value. The analysis route (`/api/analyse`) orchestrates the full pipeline: verify the session, fetch the field record, call the GEE satellite service, retrieve weather data, compute the feature vector, run ML inference, apply the four recommendation rules, persist the result to SQLite via Prisma, and return the structured JSON response.

D. GEE Integration

The backend authenticates to the GEE API using a Google Cloud service account whose private key JSON is loaded from an environment variable at startup and is never transmitted to the client or embedded in the application bundle. For each analysis request, the backend constructs an `ee.Geometry.Polygon` from the field's stored GeoJSON, filters the `COPERNICUS/S2_SR_HARMONIZED` image col-

lection to the preceding 90 days and a per-image cloud cover threshold of 20 %, applies a bitwise QA60 cloud mask to each surviving scene, and uses `ee.Reducer.mean()` to aggregate per-pixel NDVI values over the polygon geometry at 10-metre scale. The resulting list of (date, NDVI) pairs is sorted chronologically and passed to the feature engineering layer.

E. Machine Learning Pipeline

Offline training. A dataset of 610 labelled observations was assembled from agronomic literature thresholds and satellite-derived NDVI statistics. The dataset was split 80/20, yielding 488 training and 122 test samples. A scikit-learn `LogisticRegression` estimator was configured with the `lbfgs` solver, L_2 regularisation ($C=1.0$), and the multinomial multi-class strategy. Following fitting, the model's 3×5 coefficient matrix and 3×1 intercept vector were exported to `lib/ml/crop_health_model.json`.

Runtime inference. The TypeScript module `lib/ml/inference.ts` loads the JSON file once on process start-up and caches it in memory. For each analysis request it computes raw logit scores:

$$z_j = \mathbf{w}_j \cdot \mathbf{x} + b_j, \quad j \in \{\text{HEALTHY}, \text{MOD}, \text{HIGH}\} \quad (1)$$

applies numerically stable softmax normalisation:

$$P(j | \mathbf{x}) = \frac{e^{z_j - z_{\max}}}{\sum_k e^{z_k - z_{\max}}} \quad (2)$$

and returns the argmax class together with the full posterior probability vector. The entire inference path executes in a single synchronous function call with no network I/O, adding negligible latency to the analysis response time.

Recommendation engine. Following classification, the rule engine evaluates four conditions in sequence:

- **R1 (Water stress):** if `mean_ndvi < 0.35` and class is `HIGH_STRESS`, recommend urgent irrigation or moisture assessment.
- **R2 (Declining trend):** if `ndvi_trend < 0` and class is `MODERATE_STRESS`, recommend preventive intervention within seven days.
- **R3 (Stable healthy):** if `mean_ndvi > 0.60` and class is `HEALTHY`, confirm current management is adequate and suggest a 14-day re-evaluation schedule.
- **R4 (Disease / pest signal):** if `ndvi_variance > 0.02` and stress is present, recommend physical scouting for pest or disease pressure.

F. Database Implementation

The database layer uses Prisma ORM with a SQLite database file stored on the server filesystem. Prisma Migrate manages schema evolution; the current schema defines three models (`User`, `Field`, `Analysis`) and two enums (`HealthStatus`, `NDVITrend`). The NDVI time series is stored as a JSON string in the `Analysis` model's `ndviSeries` field; Prisma's `Json` scalar type handles serialisation and deserialisation transparently. SQLite was chosen for

TABLE III
MODEL PERFORMANCE ON HELD-OUT TEST SET ($n = 122$)

Class	Precision	Recall	F1-Score
HEALTHY	0.99	0.99	0.99
MODERATE_STRESS	0.94	0.96	0.95
HIGH_STRESS	0.98	0.96	0.97
Overall accuracy	0.97		

TABLE IV
FUNCTIONAL TEST CASES

ID	Feature	Scenario	Expected	Res.
TC-01	Auth	Valid credentials	Dashboard loads	Pass
TC-02	Auth	Wrong password	401 returned	Pass
TC-03	Field	Draw, save polygon	Field in DB	Pass
TC-04	Field	Area computation	Shoelace correct	Pass
TC-05	Analysis	Trigger analysis	Status badge shown	Pass
TC-06	ML	NDVI < 0.35	HS_STRESS set	Pass
TC-07	Rec.	R1 triggered	Irrigation advice	Pass
TC-08	History	Second analysis run	New record saved	Pass

prototype deployment because it requires no separate database server process; migration to PostgreSQL is possible with a single Prisma `datasource` change.

V. RESULTS AND EVALUATION

A. Model Performance

Table III reports per-class precision, recall, and F1-score on the 122-sample held-out test set. The overall classification accuracy is 97 %. The `HEALTHY` class yields the highest F1-score (0.99) as it is the most frequent and occupies a well-separated region of the feature space (high NDVI mean, near-zero trend, low variance). `HIGH_STRESS` achieves an F1-score of 0.97; its distinguishing signals are low NDVI mean combined with negative trend and elevated temperature. The most challenging boundary is between `MODERATE_STRESS` and its neighbours, reflected in the F1-score of 0.95, because the transitional NDVI range partially overlaps with both healthy late-season canopies and early high-stress conditions.

B. Functional Test Cases

Table IV summarises the functional test scenarios executed against the deployed prototype on a localhost development server. Each scenario was exercised manually by the developer following a structured test plan. All eight scenarios produced the expected outcome, confirming correct end-to-end behaviour of the authentication, field management, satellite retrieval, classification, and recommendation subsystems.

C. Non-Functional Observations

- **Responsiveness:** The full end-to-end pipeline, including the GEE API call, Open-Meteo request, and TypeScript ML inference, completed in under 4 seconds for field polygons up to 50 hectares under normal network conditions. TypeScript inference itself contributed less than 5 milliseconds of the total latency.
- **Reliability:** No application crashes or unhandled exceptions were observed during repeated end-to-end test runs on the prototype server.
- **Security:** Passwords are hashed with bcryptjs (10 salt rounds); the GEE service account private key is stored in environment variables and is never included in client-side bundles or API responses.
- **Maintainability:** The modular layer architecture allows the satellite data provider, ML model, and frontend to be updated independently without cross-layer rewrites.

VI. DISCUSSION

TobaccoGuard demonstrates that a practical, zero-licensing-cost satellite monitoring pipeline can be assembled from freely available components: the GEE Community tier, the Sentinel-2 open data archive, and a logistic regression model serialised to JSON for native TypeScript inference. The 97 % classification accuracy on the held-out test set is encouraging for a prototype, though the training data included synthetically generated observations to represent agronomic variability; external validation against field-collected, agronomist-labelled samples from actual Zimbabwean tobacco plots is the most important next step before recommending wider deployment.

The principal operational limitation is latency dependence on the GEE API. On-demand analysis requests require a synchronous GEE call that may approach web application timeout thresholds during periods of high server load. A production deployment should cache the most recent NDVI time series per field and execute daily refreshes via a background job scheduler rather than synchronous on-demand API calls. The existing modular GEE service layer already isolates this concern, simplifying the transition.

The MODERATE_STRESS class presents the hardest classification boundary to learn, as noted in the results ($F1 = 0.95$). Additional labelled observations near this boundary, combined with a supplementary spectral feature such as the Red-Edge NDVI (NDRE, computed from Sentinel-2 bands B5 and B8A), could improve class separation without requiring any change to the inference architecture, since the JSON coefficient model can simply be retrained with an extra feature column.

The four recommendation rules (R1–R4) are based on empirically derived NDVI thresholds from the remote sensing literature. While these thresholds are consistent with published stress detection ranges, they have not been calibrated against local agronomic trials in Zimbabwe. Collaboration with the Tobacco Industry and Marketing Board (TIMB) and the Department of Agricultural Technical and Extension Services (AGRITEX) would be required to validate and localise these

thresholds for the specific cultivars and growing seasons of the Zimbabwean tobacco belt.

Future development directions include: (1) NDVI spatial heat map overlays using Leaflet.heat to visualise within-field stress gradients, enabling farmers to identify problem patches rather than only field-level summaries; (2) automated daily monitoring via scheduled background jobs so that farmers receive updated results without actively triggering an analysis; (3) SMS or WhatsApp push alert delivery for farmers and extension officers who lack regular web access; and (4) expansion to other priority crops such as maize and wheat, which are equally important to Zimbabwean food security and share much of the NDVI-based classification methodology.

VII. CONCLUSION

This paper presented TobaccoGuard, a web-based tobacco crop health monitoring system that integrates Google Earth Engine satellite data retrieval, NDVI-based feature engineering, and multinomial logistic regression classification to assign one of three health statuses to any registered field polygon. The system achieved 97 % overall classification accuracy on a 122-sample held-out test set, with F1-scores of 0.99, 0.97, and 0.95 for the HEALTHY, HIGH_STRESS, and MODERATE_STRESS classes respectively, and passed all eight functional test scenarios. By executing the complete ML inference pipeline in TypeScript at runtime and leveraging the GEE free community tier, the system avoids licensing costs and eliminates Python runtime dependencies, making it practically deployable in the resource-constrained institutional environments typical of Zimbabwean agricultural support organisations. The prototype establishes a solid technical foundation for a production system capable of delivering timely, evidence-based field management advice to smallholder tobacco farmers through routine, automated satellite monitoring.

REFERENCES

- [1] Zimbabwe National Statistics Agency, “Zimbabwe Agricultural Production Report,” ZimStats, Harare, 2023.
- [2] J. Xue and B. Su, “Significant remote sensing vegetation indices: A review of developments and applications,” *Journal of Sensors*, vol. 2017, pp. 1–17, 2017.
- [3] J. W. Rouse, R. H. Haas, J. A. Schell, and D. W. Deering, “Monitoring vegetation systems in the Great Plains with ERTS,” in *Proc. 3rd Earth Resources Technology Satellite Symposium*, NASA, Washington, DC, 1974, pp. 309–317.
- [4] M. Drusch et al., “Sentinel-2: ESA’s optical high-resolution mission for GMES operational services,” *Remote Sensing of Environment*, vol. 120, pp. 25–36, 2012.
- [5] J. Delegido, J. Verrelst, L. Alonso, and J. Moreno, “Evaluation of Sentinel-2 red-edge bands for empirical estimation of green LAI and chlorophyll content,” *Sensors*, vol. 11, no. 7, pp. 7063–7081, 2011.
- [6] N. Kussul, M. Lavreniuk, S. Skakun, and A. Shelestov, “Deep learning classification of land cover and crop types using remote sensing data,” *IEEE Geoscience and Remote Sensing Letters*, vol. 14, no. 5, pp. 778–782, 2017.
- [7] D. Kpienbaareh et al., “Crop type and land cover mapping in northern Ghana using the integration of Sentinel-1 and Sentinel-2 imagery,” *Remote Sensing*, vol. 13, no. 1, p. 2, 2021.
- [8] N. Gorelick, M. Hancher, M. Dixon, S. Ilyushchenko, D. Thau, and R. Moore, “Google Earth Engine: Planetary-scale geospatial analysis for everyone,” *Remote Sensing of Environment*, vol. 202, pp. 18–27, 2017.
- [9] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

- [10] Open-Meteo, "Open-Meteo Free Weather API Documentation," 2024. [Online]. Available: <https://open-meteo.com/en/docs>
- [11] Prisma, "Prisma ORM Documentation," 2024. [Online]. Available: <https://www.prisma.io/docs>
- [12] Leaflet.js, "Leaflet: An open-source JavaScript library for mobile-friendly interactive maps," 2024. [Online]. Available: <https://leafletjs.com>